Assignment-2

ELL-786 MULTIMEDIA SYSTEMS

A report presented for the Assignment-2 on

Dictionary Techniques

Submitted By:

Sudhanshu Chaudhary (2019JTM2207)
Santosh Kumar (2019JTM2088)

Bharti School Of

Telecommunication Technology and Management

IIT, Delhi

April 2, 2021

Contents

1 Problem Statement **3**
1.1 Experiment 1: . . . 3
1.2 Experiment 2: . . . 3
1.3 Experiment 3: . . . 3

2 Theory **4**
2.1 GIF . . . 4
2.1.1 Understanding the Compression . . . 4
2.1.2 Features of the GIF format . . . 4
2.1.3 Limited color palette . . . 4
2.1.4 Dithering . . . 5
2.1.5 LZW compression . . . 5
2.1.6 Transparency . . . 5
2.1.7 Interlacing . . . 5
2.1.8 Animation . . . 5
2.2 PNG . . . 6
2.2.1 Understanding the Compression . . . 6
2.2.2 Filtering (prediction) . . . 6
2.2.3 Compression (DEFLATE) . . . 8

3 Implementation **9**
3.1 Assumptions . . . 9
3.2 Difficulties faced . . . 9
3.3 Algorithm . . . 9
3.4 Program Structure . . . 10
3.5 Screenshots . . . 11

4 Appendix **13**
4.1 Appendix-A : Code . . . 13

List of Figures

1 Filtering 6
2 PNG compression workflow 7
3 PNG vs GIF 8
4 LWZ encoding and decoding 11
5 LWZ image compression 11
6 GIF image compression 12
7 PNG image compression 12
8 LWZ, GIF, and PNG comparison 12

1 Problem Statement

This is a programming assignment which requires you to design and implement LZW encoder.

Input: A bmp file of size greater than 1 MB of your choice.

Computing Environment: Matlab / Python/ C/ C++ without using built-in libraries for LZW code.

1.1 Experiment 1:

Implement the LZW encoder and test against an initial dictionary consisting of the letters a b r y $\cdot$, then encode the following message using the LZW algorithm:
a $\cdot$ bar $\cdot$ array $\cdot$ by $\cdot$ barrayar $\cdot$ bay.
Now, convert the bmp file to a binary string and use your LZW encoder to find a compressed code. Calculate the compression ratio achieved.

1.2 Experiment 2:

Convert the bmp file to a binary string and use gif encoder to find a compressed code. Calculate the compression ratio achieved.
Study "IMAGE COMPRESSION—THE GRAPHICS INTERCHANGE FORMAT (GIF)" and compare the compression achieved by gif versus your LZW encoder. Explain additional techniques used in gif.

1.3 Experiment 3:

Convert the bmp file to a binary string and use png encoder to find a compressed code. Calculate the compression ratio achieved.
Study "IMAGE COMPRESSION—PORTABLE NETWORK GRAPHICS (PNG)" and compare the compression achieved by png versus your LZW encoder. Explain additional techniques used in gif.

Now, tabulate and compare the compression results and techniques used to compare the compression achieved by your LZW encoder versus png versus gif.

2 Theory

2.1 GIF

GIF (Graphics Interchange Format) is not a data compression method. The original version of GIF is known as GIF87a. It is graphical image format that uses variant of LZW to compress the graphical data and allows to send image between different computers. It scans the image row by row and discovers pixel correlated within row not between rows. GIF uses growing and dynamic dictionary for compressing data.

2.1.1 Understanding the Compression

- It takes number of bits per pixel b as parameter. For monochromatic image b=2 and for image with 256 colors or shades b=8.
- It uses dictionary with $2^(b+1)$ entries. At each fill up dictionary will be in double in size until 4096 entries and remain as static.
- At this point, encoder monitors the compression ratio and may decide to discard dictionary and start with new and empty one.
- At the time of taking decisions on discard, Encoder emits 2^b value as clear code which is sign for decoder to discard the dictionary.

Pointer get longer one byte from dictionary to dictionary and output are in block of 8 bytes. Each pointer preceded by header of 255 bytes maximum and terminates by bytes of 8 zeros. Pointer stores with LSB(Least Significant Bit) on left. Last block contains of value which is$2^(b+1)$.

2.1.2 Features of the GIF format

This is an overview of the different features of the GIF file format, from the perspective of a prepress operator.

2.1.3 Limited color palette

A GIF image can contain 2, 4, 8, 16, 32, 64, 128, or 256 colors which are stored in a color palette or color lookup table within the image file. Each color in the GIF color table is described in RGB values, with each value having a range of 0 to 255. CMYK colors are not possible in GIF. Although the GIF format has access to over 16.8 million colors, only a maximum of 256 can be referenced within a single GIF image. While this limited palette keeps down file size and is perfectly acceptable for on-screen viewing, it leads to posterized images when they are printed. Most pref-light tools like PitStop can generate a warning when they encounter images with a fixed color palette.

2.1.4 Dithering

The limited number of colors in GIF is used to limit the file size of images. While a small image using 256 colors may take up 9.5 K, the same image using 32 colors takes up only 4.4 K and going down to 16 colors get it down to 1.9 K. Another trick that is used to limit the file size is dithering. This technique is used to create the illusion of greater color depth by blending a smaller number of colored 'dots' together. When fewer colors are able to be displayed than are present in the original image, then patterns of adjacent pixels are used to simulate the appearance of the underrepresented colors. Dithering is not really a feature of GIF, it is simply a technique that is often used within GIF images. Dithering adds noise to the image and it reduces sharpness.

2.1.5 LZW compression

GIF supports LZW compression, which is a lossless compression algorithm that is also used frequently in prepress. TIFF images, for example, are also often LZW compressed.

2.1.6 Transparency

Transparency is the feature of the GIF89a format which allows for the specification of one of the colors in the palette to be ignored while processing the image for your display device. While this feature works great on the internet, it is not supported by layout applications, which rely on PSD-files or EPS-images with an included mask to achieve the same functionality (but with a much smoother edge around images).

2.1.7 Interlacing

Interlacing is another web-specific feature of GIF. It is a mechanism that makes images appear faster on-screen by first displaying a low-res version of the image and gradually showing the full version. Physically, an interlaced GIF just has the scan-lines stored in an unusual order:

- The first pass has pixel rows 1, 9, 17, etc (every eighth row)
- The second pass has rows 5, 13, 21, etc. (every remaining fourth row).
- The third pass has rows 3, 7, 11, 15, etc. (every remaining odd row)
- The last pass has rows 2, 4, 6, etc. (all the even-numbered rows).

2.1.8 Animation

The GIF89a specifications add a few enhancements to the file header which allows browsers such as Netscape to display multiple GIF images in a timed and/or looped sequence. This mechanism allows for small, rather crude animations and it is a very popular feature that used often used in banners. This feature is of no use for prepress software.

2.2 PNG

The PNG standard is one of the first standards to be collaboratively developed over the Internet. The impetus for it was an announcement in December 1994 by Unisys (which had acquired the patent for LZW from Sperry) and CompuServe that they would start charging royalties to authors of software that included support for GIF. The announcement resulted in an uproar in the segment of the compression community that formed the core of the Usenet group comp.compression. The community decided that a patent-free replacement for GIF should be developed, and within three months PNG was born.

2.2.1 Understanding the Compression

NG's compression process is entirely lossless; meaning that the compressed file can reconstruct the source image exactly. Done in two stages: Prediction (aka filtering) and then Compression.

2.2.2 Filtering (prediction)

Delta encoding is one of the most powerful numeric compression methods out there. The basic idea is that you can represent any value as a difference from the previous value. So, [2,3,4,5,6,7,8] can become [2,1,1,1,1,1,1],
where [2, 3–2=1, 4–3=1, 5–4=1, 6–5=1, 7–6=1, 8–7=1]
The reason this is so powerful, is that if the data is linearly correlated (that is, value has some low-difference from the previous value) then you end up transforming values of your data-set into lots of duplicate, low values, which are more compressible.
The PNG format makes use of delta encoding in a format that it calls "filtering". Basically, for each scan-line of pixels, a current pixel is encoded in some relation to the pixel to the left, the pixel above, and the pixel above-left.

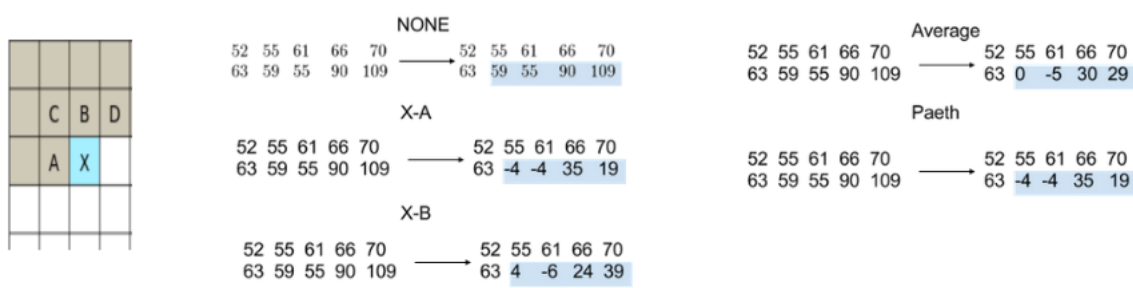


Figure 1: Filtering

The filtering takes ABC, and uses that to predict what the value of x is. The value that we replace x with, is the difference between the prediction, and the actual value.

Now, it's worth noting that each line may be a little different, which is why PNG allows 1 of 5 different modes to be chosen, per line:

- No filtering.
- Difference between X and A.
- Difference between X and B
- Difference between X and (A + B)/2 (aka average)
- Path predictor (linear function of A,B,C)

The intention here, is that each line can choose the best filtering method, for itself, such that it can produce the lowest number of unique symbols. Here's our original example with each mode: And the PNG format has some nifty ways to pick which filter to use on a channel; While brute force is the most direct, it's simply right out. Instead, the developers experimented on different image types and came up with some rules of thumb, that are close to optimal; like use None filters for palette images, and sub-8 bit gray-scale images. And for the other images, choose the filter that minimizes the sum of absolute differences; Rather than using modulo-256, instead to standard signed math, then take the abs value, and add them all together for a given row, and compare the sums for the other filter types. Choose the filter that gives the smallest sum.

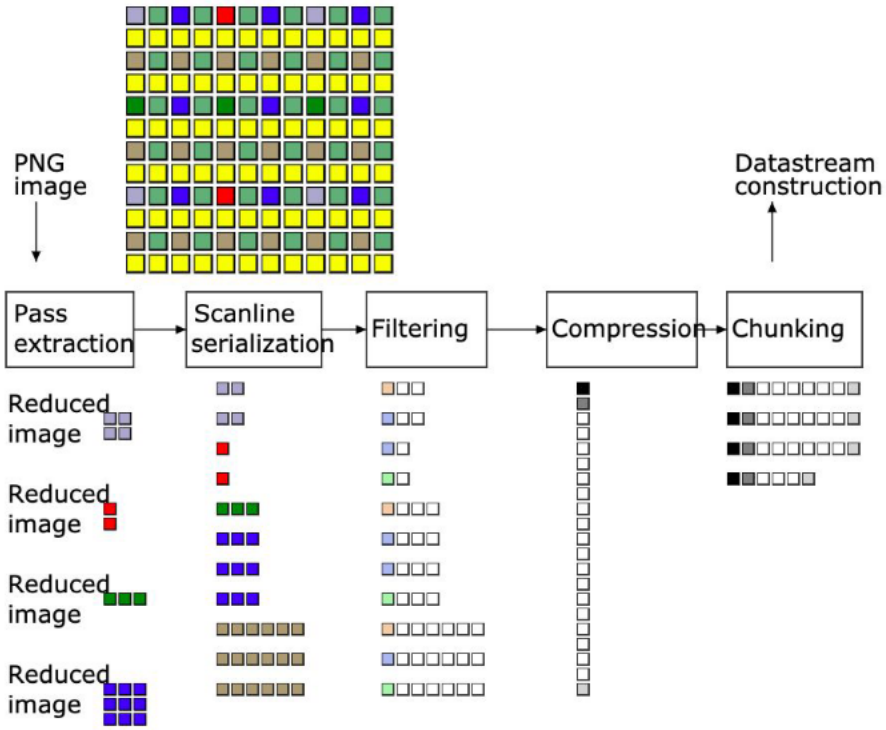


Figure 2: PNG compression workflow

2.2.3 Compression (DEFLATE)

Once filtering has occurred on a scan-line, it's passed off to a descendant of the LZ77 algorithm, known as DEFLATE; This algorithm combines LZ77 coding alongside a Huffman coder.

A compressed data set consists of a series of blocks, corresponding to successive blocks of input data. The block sizes are arbitrary, except that non-compressible blocks are limited to 65,535 bytes.

Each block is compressed using a combination of the LZ77 algorithm and Huffman coding. The Huffman trees for each block are independent of those for previous or subsequent blocks; the LZ77 algorithm may use a reference to a duplicated string occurring in a previous block, up to 32K input bytes before.

Each block consists of two parts: a pair of Huffman code trees that describe the representation of the compressed data part, and a compressed data part. (The Huffman trees themselves are compressed using Huffman encoding.) The compressed data consists of a series of elements of two types: literal bytes (of strings that have not been detected as duplicated within the previous 32K input bytes), and pointers to duplicated strings, where a pointer is represented as a pair ¡length, backward distance¿. The representation used in the "deflate" format limits distances to 32K bytes and lengths to 258 bytes, but does not limit the size of a block, except for uncompressible blocks, which are limited as noted above.

Each type of value (literals, distances, and lengths) in the compressed data is represented using a Huffman code, using one code tree for literals and lengths and a separate code tree for distances. The code trees for each block appear in a compact form just before the compressed data for that block.

A comparison between PNG and GIF for 256 x 256, 8-bit greyscale images which all require 65,536

image name	GIF (bytes)	standard PNG (bytes)	vs. GIF
bird	47516	32474	−31.7%
bridge	76511	48511	−36.6%
camera	55441	38248	−31.0%
circles	1576	1829	+16.1%
crosses	1665	2000	+20.1%
goldhill	68450	44941	−34.3%
horiz	683	693	+1.5%
lena	71115	41204	−42.1%
montage	42449	24096	−43.2%
slope	29465	11683	−60.3%
squares	931	640	−31.3%
text	4205	2339	−44.4%
total size	400007	248658	−37.8%

GIF typically 4:1 - 10:1, PNG typically 10-30% smaller than GIFs

Figure 3: PNG vs GIF

3 Implementation

3.1 Assumptions

- For LWZ dictionary be 9 bits therefore maximum size is 512.
- In GIF compression the dictionary become static at 4096.

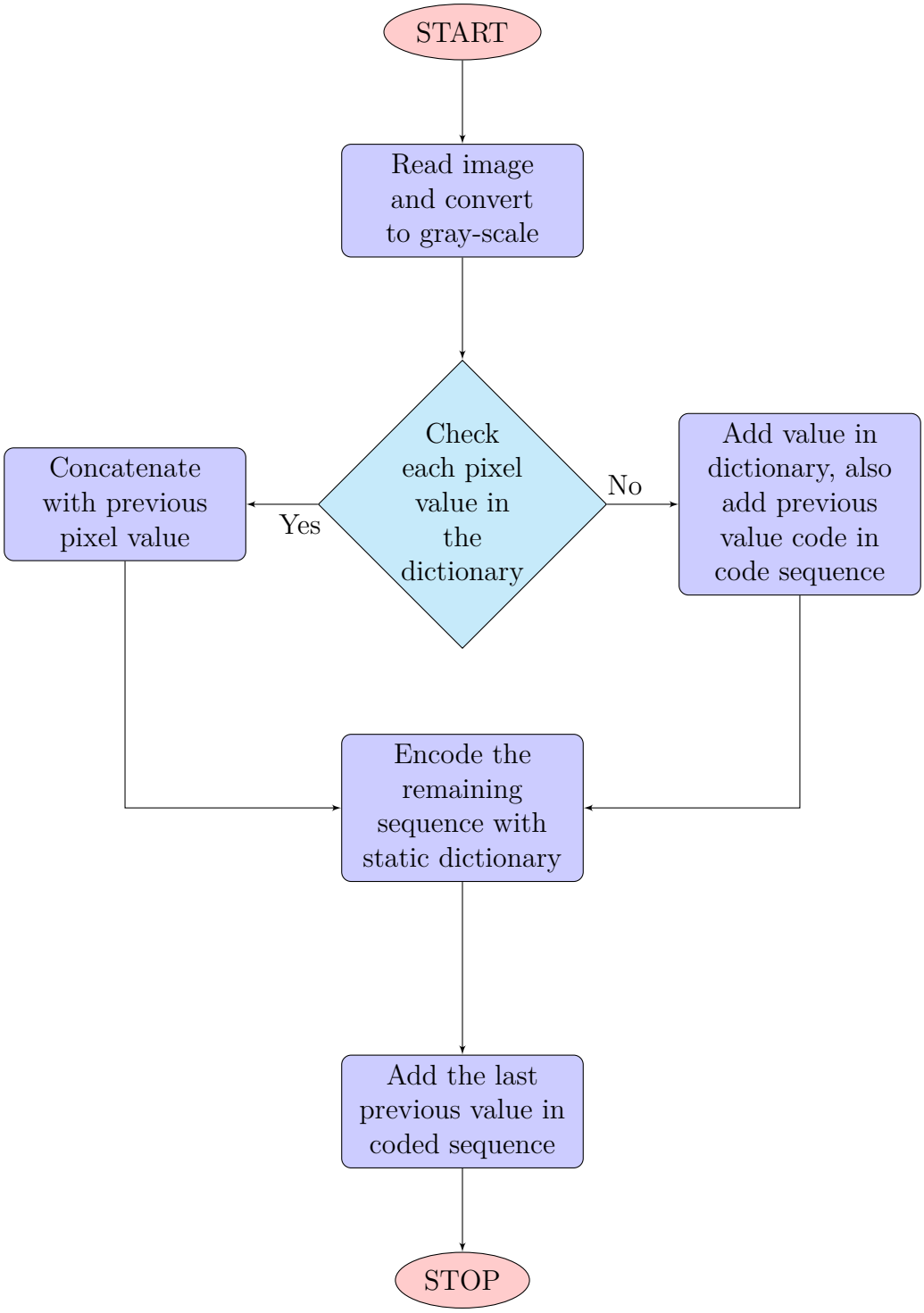
3.2 Difficulties faced

- Improving the run time of the program for high resolution images.
- Different compassion ratio for different images.

3.3 Algorithm

- Firstly, import the library needed.
- Create function for LWZ encoder and LWZ decoder.
- Now for the given dictionary and sequence, encode it using LWZ encoder.
- Decode the original sequence using decoder function from coded sequence.
- Now read the bmp image file using open CV library.
- Convert the image to gray scale.
- Now the convert the 2D image matrix to 1D and also each pixel value to string type from uint8.
- Now create a dictionary with value from 0-255 as pixel intensity.
- Encode the image using LWZ encoder and calculate the compression ratio.
- Similarly for GIF encode the image using GIF encoder and calculate the compression ratio.
- For PNG, save the png compressed image using PIL library.
- Calculate the compression ratio from original image and png compressed image.
- Compare the result from LWZ, GIF, and PNG.

3.4 Program Structure



3.5 Screenshots

```
sudhanshu@chaudhary:~$ /usr/bin/python3.6 /home/sudhanshu/Downloads/4
thSem/Multimedia/As2/A2.py
 --------------Multimedia Assignment 2---------

Dictionary given:  {'a': 1, 'b': 2, 'r': 3, 'y': 4, '.': 5}
Input string:   a.bar.array.by.barrayar.bay.
LWZ encoded val:  [1, 5, 2, 1, 3, 5, 9, 3, 1, 4, 7, 15, 8, 3, 13, 4,
9, 7, 14, 5]
LWZ dictionary val:  {'a': 1, 'b': 2, 'r': 3, 'y': 4, '.': 5, 'a.': 6
, '.b': 7, 'ba': 8, 'ar': 9, 'r.': 10, '.a': 11, 'arr': 12, 'ra': 13,
 'ay': 14, 'y.': 15, '.by': 16, 'y.b': 17, 'bar': 18, 'rr': 19, 'ray'
: 20, 'ya': 21, 'ar.': 22, '.ba': 23, 'ay.': 24}
Decoded string:  a.bar.array.by.barrayar.bay.
```

Figure 4: LWZ encoding and decoding

```
sudhanshu@chaudhary:~$ /usr/bin/python3.6 /home/sudhanshu/Downloads/4
thSem/Multimedia/As2/A2.py
 --------------Multimedia Assignment 2---------

Image size:  1420419
LWZ coded sequence size:  1327769
LWZ dictionary size:  512
Compression Ratio LWZ:  1.0697787039763693

Execution time:  0:00:02.436074
```

Figure 5: LWZ image compression

```
sudhanshu@chaudhary:~$ /usr/bin/python3.6 /home/sudhanshu/Downloads/4
thSem/Multimedia/As2/A2.py
 --------------Multimedia Assignment 2---------

Image size:  1420419
GIF coded sequence size:  827148
GIF dictionary size:  4096
Compression Ratio GIF:  1.7172489082969433

Execution time:  0:00:03.495298
```

Figure 6: GIF image compression

```
sudhanshu@chaudhary:~$ /usr/bin/python3.6 /home/sudhanshu/Downloads/4
thSem/Multimedia/As2/A2.py
 --------------Multimedia Assignment 2---------

Orignal image size(bytes):  4264316
PNG image size(bytes):  1952824
Compression Ratio PNG:  2.1836663211840905

Execution time:  0:00:04.465210
```

Figure 7: PNG image compression

```
sudhanshu@chaudhary:~$ /usr/bin/python3.6 /home/sudhanshu/Downloads/4
thSem/Multimedia/As2/A2.py
 --------------Multimedia Assignment 2---------

Image size:  1420419

Compression Ratio LWZ:  1.0697787039763693

Compression Ratio GIF:  1.7172489082969433

Compression Ratio PNG:  2.1836663211840905

Execution time:  0:00:04.382014
```

Figure 8: LWZ, GIF, and PNG comparison

4 Appendix

4.1 Appendix-A : Code

```python

# ——————————Multimedia assignment 2————————
print(" ——————————Multimedia Assignment 2————————")
print()
# Library imported
import os
import cv2
import time
import numpy as np
from PIL import Image
from datetime import timedelta

# start timer to check time
start_time = time.monotonic()

path = "/home/sudhanshu/Downloads/4thSem/Multimedia/As2/1.bmp"
pathout = "/home/sudhanshu/Downloads/4thSem/Multimedia/As2/"

##########################################################
# LWZ encoder function
def LZWencoder(seq, table):
    tsize = len(table) + 1
    encodedSeq = []
    prev_val = ""
    i = 0
    while i < len(seq):
        curr_val = str(seq[i])
        comp_val = prev_val + curr_val
        if comp_val in table:
            prev_val = comp_val
        elif len(table) == 512:
            encodedSeq.append(table[prev_val])
            prev_val = curr_val
        else:
            # print(table[prev_val])
            encodedSeq.append(table[prev_val])
            table[comp_val] = tsize
            tsize += 1
            prev_val = curr_val
        i += 1
    if i == len(seq):
        encodedSeq.append(table[prev_val])
    # print("Code length: ", len(encodedSeq))
    # print("Dictionary length: ", len(table))
    return encodedSeq, table


##########################################################

```

```python
# LWZ decoder function
def LZWdecoder(code, dic):
    decodeSeq = ""
    temp = ""
    for i in code:
        if len(dic) >= i:
            decodeSeq += dic[i - 1]
            temp += dic[i - 1][:1]
            if temp not in dic:
                dic.append(temp)
                temp = temp[-1]
            temp += dic[i - 1][1:]
    # print(decodeSeq)
    return decodeSeq


#######################################################
# GIF encoder function
def GIFencoder(seq, table):
    table["clear"] = len(table)
    table["end"] = len(table)
    tsize = len(table)
    encodedSeq = [table["clear"]]
    prev_val = ""
    i = 0
    while i < len(seq):
        curr_val = str(seq[i])
        comp_val = prev_val + curr_val
        if comp_val in table:
            prev_val = comp_val
        elif len(table) == 4096:
            encodedSeq.append(table[prev_val])
            prev_val = curr_val
        else:
            encodedSeq.append(table[prev_val])
            table[comp_val] = tsize
            tsize += 1
            prev_val = curr_val
        i += 1
    if i == len(seq):
        encodedSeq.append(table[prev_val])
    encodedSeq.append(table["end"])
    # print("Code length: ", len(encodedSeq))
    # print("Dictionary length: ", len(table))
    return encodedSeq, table


#######################################################
# GIF decoder function
def GIFdecoder(code, dic):
    dic.append("clear")
    dic.append("end")
    # print(dic)
    decodeSeq = ""
```

```
    temp = ""
    code = code[1:]
    for i in code:
        if len(dic) >= i:
            decodeSeq += dic[i - 1]
            temp += dic[i - 1][:1]
            if temp not in dic:
                dic.append(temp)
                temp = temp[-1]
            temp += dic[i - 1][1:]
    decodeSeq = decodeSeq[:-1]
    # print(decodeSeq)
    return decodeSeq


#######################################################

dictionary = {"a": 1, "b": 2, "r": 3, "y": 4, ".": 5}
orignaldict = ["a", "b", "r", "y", "."]
inputstring = "a.bar.array.by.barrayar.bay."

print("Dictionary given: ", dictionary)
print("Input string:  ", inputstring)
a, b = LZWencoder(inputstring, dictionary)
c = LZWdecoder(a, orignaldict)
print("LWZ encoded val: ", a)
print("LWZ dictionary val: ", b)
print("Decoded string: ", c)
print()
#######################################################

img = cv2.imread(path)  # read image
img1 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # convert to grayscale
m, n = np.shape(img1)
img2 = img1.flatten().astype(str)  # convert to 1D
print("Image size: ", m * n)
print()
gray_dictionary = dict((str(i), i) for i in range(256))
compress_img, g_dic = LZWencoder(img2, gray_dictionary)
print("LWZ coded sequence size: ", len(compress_img))
print("LWZ dictionary size: ", len(g_dic))
print("Compression Ratio LWZ: ", len(img2) / len(compress_img))
print()
#######################################################

compress_gif, gif_dic = GIFencoder(img2, dict((str(i), i) for i in range(256))
    )
print("GIF coded sequence size: ", len(compress_gif))
print("GIF dictionary size: ", len(gif_dic))
print("Compression Ratio GIF: ", len(img2) / len(compress_gif))
print()

#######################################################
Image.open(path).save(pathout + "op.png")
```

```
s1 = os.path.getsize(path)
s2 = os.path.getsize(pathout + "op.png")
print("Orignal image size(bytes): ", s1)
print("PNG image size(bytes): ", s2)
print("Compression Ratio PNG: ", s1 / s2)
print()

#########################################################
end_time = time.monotonic()
print("Execution time: ", timedelta(seconds=end_time - start_time))
print()
#########################################################
```

References

[1] *Compression and Filtering*. http://www.libpng.org/pub/png/book/chapter09.html.

[2] *GIF*. https://en.wikipedia.org/wiki/GIF.

[3] *LZW Compression*. https://www.dspguide.com/ch27/5.htm.

[4] *What's In A GIF*. http://giflib.sourceforge.net/whatsinagif/lzw_image_data.html.